

CSE314: Compiler Design Lab Assignments [Spring 2026]

[Provided on 15/3/26]

***** Any copying of any submission will result in 0 marks, for all groups / individuals involved *****

Submissions:

1. **Project source code**- max 20 marks (Groupwise) **Deadline: 10 April for all section**
2. **Project report** (about how you solved the task of the project) -max 10 marks (Groupwise)
Deadline: 10 April for all section
3. **Lab Report** [**part 1**: Two examples of moderate size, of combined NFA to DFA conversion and screenshot of code implementation with correct and wrong input ,
part 2: Three examples of moderate size CFG to first follow table and LL(1) parsing table construction and code implementation with right and wrong example input, in both part, show screenshot of code output for valid and invalid input] - **max 25 marks** (Individual).
Submission date:
Soft copy: 31 March all section
Hard copy: 1st class after Eid vacation.
4. **Viva(10 marks)** **dates: From 14 April as per class schedule.**

Instructions:

You are tasked to design the lexer and parser in C for a custom programming language with rules and restrictions as given below,

Rules:

1. All programs start with **#include<stdio.h>** . It's a fixed line. This line must be the beginning of the code
2. There can be single line comments anywhere from the second line onwards. **Comments can contain only space and alphabets**, nothing else.
3. Variable names **must start with an underscore**, then one or more alphabets and then a one digit number followed by just one alphabet. Ex: **_result4m**
4. Function names must end with **Fn** and only alphabets can be used in its name. **computeValueFn**
5. **..** has to be used to end any statement.
6. Variable and function return type can be either **int or dec**. Variables or numbers can be returned.
7. Loops must start with a pattern like **loop_main01**: where the part between **_** and **:** has to be one or more alphabets followed by two digits. only while loop exists and it deals with

only a single variable. It accepts variable type, variable name then the only comparator sign < and then one or more digit numbers followed by ..

8. There is one built-in function **printfFn** with structure like **printfFn(_result4m)**, with variable name that follows variable naming rule inside ().
9. Custom functions like **computeValueFn(dec_val1a) []** may or may not be present. But if they do, they will just accept one variable type followed by a variable name as input. Custom functions can be one or more and must be placed after the first line of the code and before the main function.
10. **MainFn** function is a mandatory function that must exist.

1. To design your final code, you must use the lexer and parser code provided by me during the lab with minor modifications only that's essential for the task. No other code will be accepted. It's not just about creating solution but also about only using specified resources for it

2. You must keep the chat history with the AI so that your source of code can be verified. Do not use a tool where this cannot be done. Without this your project will not be accepted. I will check it from your device before proceeding further.

3. Maintain deadlines.

Input will be the codes that follow the rules, output will have two parts. Lexical part showing translation of elements of code to tokens and second part to show parsing of the tokenized output of first part and final verdict whether the program was accepted or not.

Usage of modern tools like ChatGPT is highly encouraged.

For further clarifications of the project requirements, a video link is provided here.



Lexer and Parser Design...

Valid program example for testing:

Example 1:

```
#include<stdio.h>

//single line comment

dec computeValueFn(dec_val1a) [
```

```

    dec _temp2x = _val1a + 5..
    return _temp2x..
]

int mainFn() [
    dec _input3k = 10..
    dec _result4m = computeValueFn(_input3k)..

    loop_main01:
    while (dec _loopin0x < 3..) [
        printfFn(_result4m)..
        break..

    ]

    return 0..
]

```

Example 1 v2 :

```

#include<stdio.h>

//single line comment

dec computeValueFn(dec _val1a) [
    dec _temp2x = _val1a - 5..
    return _temp2x..
]

int mainFn() [
    dec _input3k = 10..
    dec _result4m = computeValueFn(_input3k)..

    loop_main01:
    while (dec _loopin0x <= 3..) [
        printfFn(_result4m)..
        break..

    ]

    return 0..
]

```

Example 2:

```
#include<stdio.h>

int mainFn() [
    dec _input3k = 10..
    dec _result4m = 2..

    loop_main01:
    while (int _m7x < 3..) [
        printfFn(_rt4c)..
        break..

    ]

    return 0..
]
```

Example 3:

```
#include<stdio.h>

int mainFn() [
    dec _input3k = 10..
    dec _result4m = 2..
    printfFn(_rt4c)..
    return 0..
]
```